PRINCETON UNIVERSITY PRESS

Chapter Title: A. BASIC ELEMENTS OF THE PYTHON PROGRAMMING LANGUAGE

Book Title: Natural Complexity

Book Subtitle: A Modeling Handbook

Book Author(s): Paul Charbonneau

Published by: Princeton University Press

Stable URL: https://www.jstor.org/stable/j.ctt1vwmgzt.16

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at https://about.jstor.org/terms

Princeton University Press is collaborating with JSTOR to digitize, preserve and extend access to *Natural Complexity*

JSTOR

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms

A

BASIC ELEMENTS OF THE PYTHON PROGRAMMING LANGUAGE

This appendix is *not* meant to be a comprehensive introduction to the Python programming language. It aims only at presenting—and sometimes providing additional explanations regarding—the use (and possible misuse) of the basic elements of Python on which the codes presented throughout this book are built. The developers of Python and assorted Python libraries have done a pretty superb job of providing online documentation, URLs for which are given at the end of this appendix. Also, never hesitate to google a Python query, you are very likely to get the answer you need (and then some).

With a few exceptions, only syntax elements common to most computing languages are used throughout this book, to ease translation for those wanting to work with a computing language other than Python; going all-out Python could have made many coding constructs more elegant and compact, and run faster, but also made them harder to decipher for non-Python-savvy users. This being said, I found a few Python-specific constructs so useful that I ended up using them; in all cases, their functionality is explained in what follows, and alternative code fragments omitting their use are also provided.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms

Raw Python is actually a pretty minimal language for the purpose of numerical computation, but these limitations are readily bypassed by the use of various Python libraries. All Python codes provided in the chapters of this book use functions from the NumPy library. Python libraries are still rapidly evolving, but at this point in time, NumPy is a standard. I generally steered clear of high-level functions for scientific computation, to facilitate portability to other computing languages. If you think you need those, look into the SciPy library.

A.1 Code Structure

Python is really a scripting language, so that Python source codes, which should really be called "scripts," are not "compiled" but rather "interpreted." The distinction may be profound from the computer science point of view, but for the user, the most important consequence is that a Python source code is interpreted strictly sequentially, line by line.

Coding certainly carries one's style, and there is definitely satisfaction to be had in writing elegant source code. Style may be personal, but some standard, good programming habits should also be adopted. For example, judicious definition of functions as program subunits is very good programming practice, as it often helps to highlight the overall logic of the code, and favors code modularity. The so-called PEP8 style guide for Python code (see the URL at the end of this appendix) offers useful guidelines on nearly all aspects of Python programming.

A typical self-contained Python source code, such as those presented throughout this book, is structured like this:

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms

```
# ONE OR MORE COMMENT LINES EXPLAINING BRIEFLY WHAT THIS CODE DOES
import numpy as np              # import (and rename) NumPy library
import matplotlib.pyplot as plt # import (and rename) matplotlib library
...
#-----------------------------------------------------------------------
PI=3.1415926536                 # define constants
...
#-----------------------------------------------------------------------
# THIS IS A USER-DEFINED FUNCTION
def my_first_function(x,y):
    ...                         # Python instructions calculating z
    return z
# END FIRST FUNCTION

# THIS IS ANOTHER USER-DEFINED FUNCTION
def my_second_function(z):
    ...                         # Python instructions calculating s
    return s
# END SECOND FUNCTION
#-----------------------------------------------------------------------
# MAIN PROGRAM
...                             # assorted Python instructions,
z=my_first_function(x1,y1)      # including calls to my_first_function
# END
```

For a more complete working example see the hodgepodge code listing in figure 11.4. A few things are worth nothing here:

1. You'll have guessed already that `#` is the comment character in Python. Adding descriptive comments to lines of code is always a good idea. Leaving a line blank has no impact on code execution, but can improve readability, which is also a good idea. In codes of significant length, I like to separate functions from constant definitions, programs, etc., by a comment line of `---`, but that's just me.
2. It is often practical to rename libraries upon import; here, for example, the NumPy library is internally renamed `np`, so that invoking the NumPy function `array()` can now be done as `np.array()` instead of `numpy.array()`. The advantage of such renaming is

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms

perhaps more apparent when invoking functions from a library with a longer name, for example matplotlib.pyplot.

3. The set of instructions associated with a function is identified *only* by being indented to the right with respect to the `def` header. I highly recommend adding a comment line, as in the above example, to explicitly flag the end of instructions lines pertaining to a function.
4. User-defined Python functions can return more than one value via their `return` statement; see, for example, the `return fx,fy` instruction in the `force` function in figure 10.3.
5. In Python user-defined functions, the `return` statement is optional; functions can operate internally on their arguments, which are then modified upon terminating the function.
6. If you opt to lump everything into a single source file, function definitions must appear *before* being invoked the first time. Otherwise source code for functions must be run through Python prior to running the main program. Consequently, in the above global code structure example, the second user-defined function could call the first, but not the other way around.
7. Any variable declared and initialized prior to the definition of functions and the main program, such as the constant `PI` in the above code fragment, is *global* and as such can be used (but not modified) by any program subunit that follows.
8. Unlike functions, the main program need not be assigned a name. Nostalgic C programmers (like me), desperately longing for `int main(void)`, can include a comment line with `MAIN`, as in the above example.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms

Python's NumPy library includes the usual set of predefined mathematical functions such as `sin()`, `cos()`, `log()`, and `sqrt()`. If you need it, it most likely exists within Python. Typing the first thing that comes to mind will often get you what you want, otherwise simply fall back on the online documentation.

A.2 Variables and Arrays

Python supports the usual variable types: float, integer, character, Boolean, etc. Variable types need not be explicitly defined, i.e., in interpreting the instruction `a=1` Python will assign integer status to the variable `a`, but would assign floating-point status if the instruction had read `a=1.0`.

In Python, variable and function names can be longer than you might ever want them to be. Lower- and uppercase characters are distinct, i.e., variables named `aa` and `AA` are not the same objects. As in all computing languages, Python reserves certain character strings as keywords for its own internal purposes. You won't need much Python programming experience to figure out that naming a variable `for`, `if`, `else`, `def`, `return`, etc. is probably not a good idea. Some reserved Python keywords are not as intuitively obvious; once upon a time I got into trouble naming a variable `del`, the Python keyword used to delete an element from a list, and the error message I received was not exactly transparent. Other nonintuitive keyword names, all to be avoided as variable names, include `break`, `class`, `in`, `is`, `lambda`, `nonlocal`, `pass`, `try`, and `yield`. To this blacklist must be added the character strings identifying Python's predefined functions such as `min`, `max`, and `range`.

Raw Python supports lists, but the fixed length arrays commonly used in numerical computation are created through specific functions in the NumPy library. Only three are used in

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms

this book, and what they actually do depends on the argument provided. For example,

1. `dx=np.array([-1,0,1,0])` creates a 1-D array of length 4 named `dx`, containing the four integer values -1, 0, 1, 0;
2. `grid=np.zeros([N,M])` creates a 2-D array named `grid` of size $N \times M$, i.e., of length N in the first dimension (rows/vertical) and M in the second (columns/horizontal) and fills it with the (float) value zero—very useful for initialization (the default variable type is float);
3. `status=np.ones(M, dtype='int')` creates a 1-D array of length M named `status`, and fills it with the integer value 1—also very useful for initialization.

Arrays can also be defined implicitly through mathematical operations, or the `return` of a function. For example, if `a` has already been defined as a 1-D array of length N, the instruction `b=a` will create an second array `b` of length N and fill it with the corresponding elements of `a`. Likewise, in the code fragment presented above, if `z` returned by the first function is a 2-D array of size $N \times N$, the instruction

```
q=my_first_function(x,y)
```

will create a 2-D $N \times N$ array named `q` and fill it with the elements of the local array `z` calculated internally within that function.

Individual array elements are accessed through their *index*, giving their position within the array. Python numbers elements of an array of length N from 0 to $N-1$, so that `a[1]` accesses the *second* element of array `a`, `a[N-1]` the last, and `a[N]` will blow you out of array bounds. You get used to it eventually.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms

A.3 Operators

Python includes all the basic arithmetical operators, using the usual keyboard symbols `+`, `-`, `*`, `/` for addition, subtraction, multiplication, and division, respectively. Note that, unlike in many computing languages, in Python explicit integer division such as `7/2` will return 3.5, rather than 3; in other words, implicit conversion to real type will take place. If you really want to divide two integers and get a truncated integer result, you must use Python's integer division operator `//`; for example, `7//2` will return the integer 3.

Python allows very flexible use of the value assignment operator `=`, for example the one-line instruction `a,b,c=0,1.,0` sets `a=0` (integer), `b=1.0` (float), and `c=0` (integer). Powers use the old FORTRAN syntax `**`, i.e., `a**2` is the same as `a*a`, and fractional exponents are allowed, so that, for example, `a**(1/3)` returns the cube root of `a`.

Python also includes many other arithmetical operators, some quite useful, for instance the additive/multiplicative increment/decrement operators `+=`, `-=`, `*=`, and `/=`, where

`a+=b` is equivalent to	`a=a+b`,	`a-=b` is equivalent to	`a=a-b`,
`a*=b` is equivalent to	`a=a*b`,	`a/=b` is equivalent to	`a=a/b`.

Another very useful operator is the modulus `%`, such that `a%b` returns the remainder of the division of (positive) integer `a` by (positive) integer `b`. This is particularly useful to enforce periodicity to random walks on lattices (see, e.g., the ant code listed in figure 2.10). Consider the instruction

```
ix=(N+ix) % N
```

As long as $0 \leq \texttt{ix} < \texttt{N}$, then $\texttt{N} \leq \texttt{N} + \texttt{ix} < 2\texttt{N}$, so that the above instruction will leave the value of `ix` unchanged; but if $\texttt{ix} < 0$, then this instruction will add `N` to `ix`; and will subtract `N` if $\texttt{ix} \geq \texttt{N}$.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms

Under Python's NumPy module, arithmetical operators can also act on arrays. For example, if `a` and `b` are two 1-D arrays of length `N`, the instruction

```
c=a+b
```

creates an array `c` also of length `N`, and sets its elements equal to the pairwise sum of the elements of `a` and `b`. This is equivalent to the following instructions:

```
c=np.zeros(N)
for i in range(0,N):
    c[i]=a[i]+b[i]
```

This works only if the arrays have the same dimensions and lengths, otherwise Python will return a run-time error. However, one useful Python/NumPy-legal possibility, used in this book, is to add a scalar to every element of an array. For example, in the earthquake code of figure 8.3, lattice driving takes place by adding the same scalar increment `delta_f` to every element of the 2-D array `force` through the instruction

```
force[:,:]+=delta_f
```

where the `:` symbol signifies "all elements of this array dimension."[1]

A.4 Loop Constructs

Python supports the usual two basic loop constructs: fixed-length (**for**) loops and conditional (**while**) loops. The basic syntax for a

[1]The shorter instruction `force+=delta_f` would be Python-legal as well, and achieve the same result. I find this to be potentially confusing when reading the code, and so such syntax is avoided everywhere in this book.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms

fixed-length loop is the following:

```
for i in range(0,N):
    ...
```

where ... stands for one or more lines of syntactically correct Python instructions. This loop would repeat N times, with the loop index variable i running from 0 to $N-1$; i.e., *not* from 0 to N, as the colloquial meaning of "`range`" would normally suggest. A third, optional parameter can be provided to `range`, controlling the size of the increment for the loop control variable; writing the above as `for i in range(0,N,2)` would run the loop with values $i = 0, 2, 4, 6, 8, \ldots, N-1$ (or $N-2$ if N is even). It takes a little while to get used to this convention, but it works, and has at least the merit of being compatible with array indexing, in which elements of an array of length N are also indexed from 0 to $N-1$.

The fixed-length loop structure just described runs over a preset number of iterations, determined by the two arguments given to `range()`. In some situations it might not be possible to determine a priori the number of iterations required by a loop. For example, in the DLA simulations of chapter 3, the temporal iteration only needs to run until all particles are stuck, or, in the epidemic simulation of chapter 9, until the number of infected individuals has fallen to zero. The appropriate temporal loop construct in such a case would be, as in the epidemic code of figure 9.1,

```
max_iter=100000         # maximum number of iterations
...
iterate=0               # iteration counter
while (n_infect>0) and (iterate<max_iter): # temporal loop
    ...                 # line(s) of Python instructions
    iterate+=1          # increment iteration counter
# end of temporal loop
```

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms

Note that the loop control condition includes a safety test ensuring that the loop cannot run forever, if some algorithmic design flaw or coding mistake were to cause `n_infect` to never fall to zero.[2]

Something equivalent to conditional loops can also be constructed using the `break` instruction, which prematurely exits an ongoing loop and picks up execution with the first instruction following the end of the loop. As a specific example, the conditional loop above could be written instead as

```
max_iter=100000            # maximum number of iterations
...
for iterate in range(0,max_iter): # temporal loop
   ...                     # line(s) instructions
   if n_infect == 0: break # break out of loop prematurely
# end of temporal loop
```

Such use of the `break` statement to build conditional loops is often not considered good programming style, but it can be useful in some circumstances.

A particularly objectionable (IMHO) feature of loop syntax in Python is that the block of instructions acted upon by the loop is identified *only* by being indented with respect to the loop instruction, which means, for example, that

```
for i in range(0,N):
    a=i+1
    print("a= {}.".format(a))
```

will *not* produce the same output as

```
for i in range(0,N):
    a=i+1
print("a= {}.".format(a))
```

[2]I highly recommend developing this to a reflex when coding **while** loops.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms

In the first case, the value of `a` would be printed to screen at every iteration of the loop, but in the second case only the last value would, after exiting the loop. If the loop controls only a few lines of instructions, this indentation-based loop-delimiting syntax is tolerable; but for loops containing many instructions, or other nested loops, or conditional blocks of instructions, the code logic can become harder to follow. As a compromise, in many of the codes listed in this book, I have added a comment line to explicitly mark the end of long instruction blocks associated with loops or conditional statements, as exemplified in the **while** loop example above.

Note finally that if a loop controls a single line of instruction, all of this can be written on the same line, as in line 6 in the box-count code of figure 3.10:

```
while (2**n_scales<n) and (n_scales<100): n_scales+=1
```

Like many modern programming languages, Python also supports a form of implicit loop defined using the symbol `:`, used to access a subset of contiguous array elements. For example, if `A` is an array of dimension 1 and length N, writing `A[i1:i2]` accesses elements `i1` to `i2-1` of the array. This may seem straightforward, but where it becomes potentially confusing is in a statement like `A[0:10]`, which accesses the 10 elements indexed from 0 to 9 of array `A`. To make things worse, *this convention is different in many other computing languages*, where the equivalent of the Python `A[0:10]` syntax would mean "access array elements indexed 0 through 10 inclusively," now for a total of 11 elements. Seasoned MATLAB and IDL programmers, beware!

This being said, the syntactic shortcuts allowed by the use of `:` are just too useful to skip. Consider, for example, the following instruction in the source code for cluster tagging listed in figure 4.3:

```
map_cluster[1:N+1,1:N+1]=lattice[:,:]
```

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms

In a single instruction line, this copies the $N \times N$ array `lattice` into the larger $(N+2) \times (N+2)$ array `map_cluster`, leaving all edge values (ghost nodes) of `map_cluster` at their initialized zero value. This instruction is thus equivalent to the double loop construct

```
for i in range(0,N):
    for j in range(0,N):
        map_cluster[i+1,j+1]=lattice[i,j]
```

A.5 Conditional Constructs

Python includes all the usual **if**... and **if**...**else**... conditional constructs, with logical conditions expressed in terms of the (self-explanatory) operators <, >, <=, >=, as well as the somewhat less self-explanatory == and != for "equal to" and "not equal to," respectively. Conditional statements constructed in this manner can be combined using the usual `and` and `or` logical operators. Two examples should suffice to illustrate the concept, the first taken from the epidemic code of figure 9.1. The block of instructions following

```
if (infect[k] == 0) and (k != j):
```

is executed provided both conditions within parentheses are satisfied (i.e., evaluate to Boolean TRUE).[3] The second example is taken from the earthquake code of figure 8.3:

```
if toppling[iterate] > 0:
    force+=move
else:
    force[:,:]+=delta_f
```

[3]The parentheses "`(...)`" are optional but I highly recommend their use in such compound conditional expressions.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms

Here the 2-D lattice `force` is updated by addition of the 2-D array `move` if at least one toppling has occurred (first block of instructions), otherwise the scalar increment `delta_f` is added at every node of the lattice (second block of instructions). As with loop constructs, the blocks of instructions controlled by the conditions are *only* delimited by being indented to the right (I really hate this!), but a single condition-controlled instruction can be included after the colon as a single instruction.

Python does not include a straightforward **case** (or **switch**) construct; these must be built using sequential **if** or nested **if**. . . **else** statements, or the Python contracted version **elif**. See the lattice update rules in the hodgepodge code listed in figure 11.4 for a specific example.

One type of Python-specific conditional instruction is so useful that I opted to make use of it in some of the codes listed in this book. For example, in figure 4.3,

```
if iic in map_cluster[jj+dx[:]],kk+dy[:]]:
    ... # instruction(s) subject to conditional execution
```

This searches for the presence of the value `iic` in any one of the nearest neighbors of node `[jj,kk]`, as defined by the four element pairs stored in the template arrays `dx` and `dy`. This single instruction is here equivalent to the construct

```
ifound=0
for i in range(0,4):
    if map_cluster[jj+dx[i],kk+dy[i]] == iic:
        ifound=1
if ifound == 1:
    ... # instruction(s) subject to conditional execution
```

A.6 Input/Output and Graphics

Python includes the usual set of functions for writing or reading to files, printing to screen, or reading keyboard input. The only one used in the codes listed throughout this book is the basic "print to screen" function `print`; see, for example, line 43 in the DLA code of figure 3.1 for a specific example. "Pretty printing,"

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms

with full control over format, is of course possible. See the Python documentation for more on all this I/O stuff.

The output of most simulations described throughout this book is usually best displayed as pixelated images (for any simulation defined over a lattice), or even better, animation of such images. The Python library matplotlib contains many user-friendly graphical functions that do exactly this. See the URL provided below. Most codes included in this book use only very basic plotting instructions, all using matplotlib.

Some simulations, for example the forest-fire model of chapter 6, the flocking simulations of chapter 10, or the hodge-podge spiral simulations of chapter 11, are most definitely best appreciated as animations; unfortunately, there is as yet no really easy way to do this in Python. The closest is the matplotlib function `funcAnimation`, but it requires encapsulating the simulation time steps as a function to be called by the animation function. I opted to leave this out of the codes provided throughout this book, but it is definitely worth the effort. Note also that precomputed animation files for some of these simulations are available via the Princeton University Press website at

http://press.princeton.edu/titles/11053.html.

A.7 Further Reading

The official home page of the Python programming language is

www.python.org.

It gives (free) access to the software, provides download and installation instructions, user's guides, and beginner's tutorials, as well as many code examples. Their Python tutorial contains pretty much everything you need to know (and a lot more) to work through this book:

www.pythonprogramminglanguage.com.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms

The PEP8 style guide is also available here:

www.python.org/dev/peps/pep-0008.

Two other excellent Python resources are

Langtangen, H.P., *A Primer on Scientific Programming with Python*, 4th ed., Springer (2014);
Swaroop, C.H., *A Byte of Python*, www.swaroopch.com/notes/python/,

as well as the Python tutorial from Code Academy:

www.codeacademy.com.

For programming beginners, the Python tutor is excellent:

www.pythontutor.com.

On the NumPy and matplotlib Python libraries, see

www.numpy.org,
www.matplotlib.org.

At this writing, the easiest way to get started downloading and installing these (and other) Python libraries is through either one of the following open platforms:

www.scipy.org,
www.continuum.io.

No point procrastinating, start downloading now!

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:46:49 UTC
All use subject to https://about.jstor.org/terms